COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d

SWINBURNE UNIVERSITY OF TECHNOLOGY

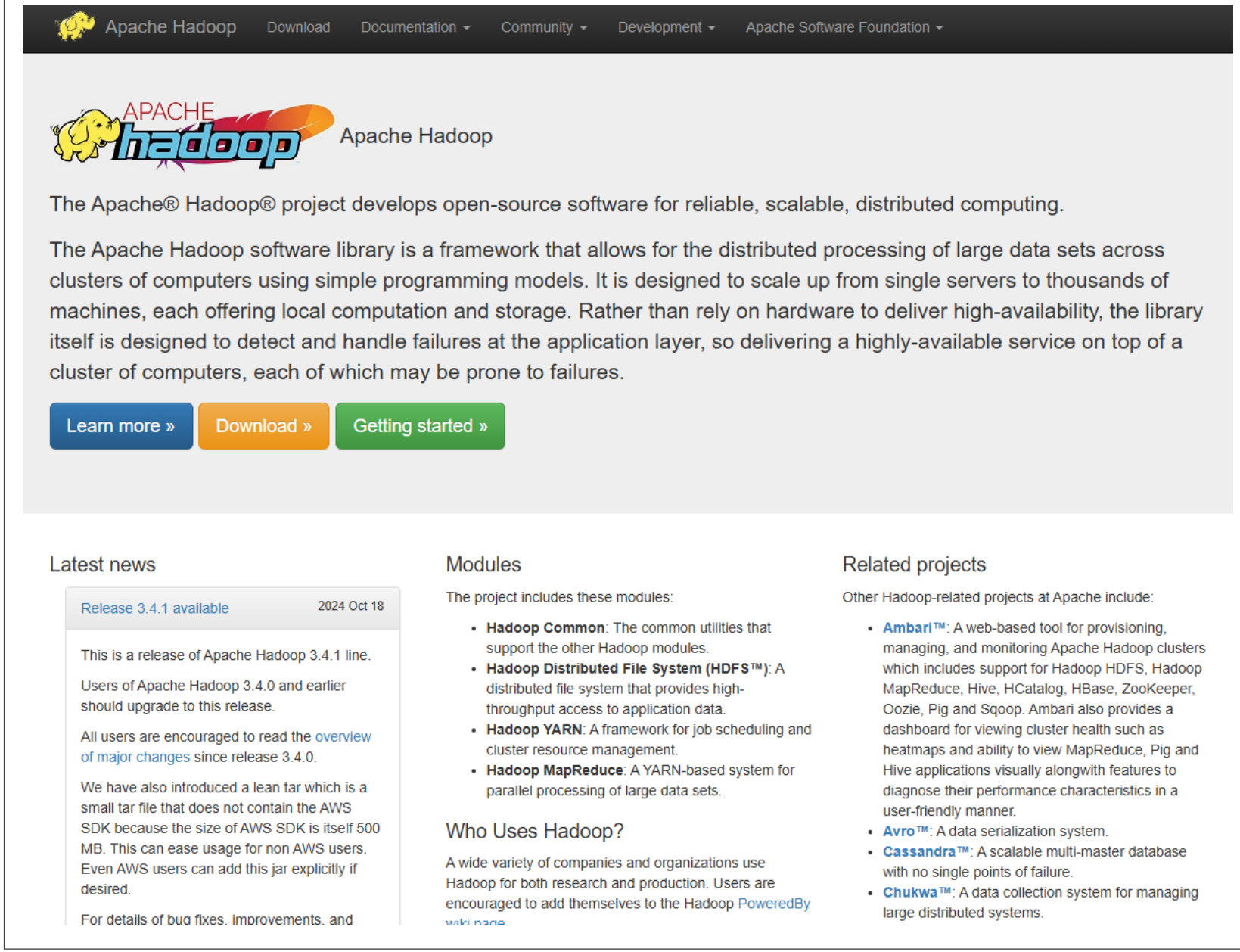
Week 4

Writing MapReduce Program -II

GOING DEEPER INTO THE HADOOP API

Hadoop API Current Version

- Apache Hadoop Main 3.4.1 API



Why Use ToolRunner?

You can use *ToolRunner* in MapReduce driver classes.

- This is not mandatory, but it is the best practice.

ToolRunner uses the *GenericOptionsParser* class internally.

- Specify configuration options.
- Specify items for the Distributed Cache.

DRIVER CODE

To implement the ToolRunner, simply complete the driver code.

Driver Code

- Import the relevant classes.

```
1  package stubs;
2
3  import org.apache.hadoop.fs.Path;
4  import org.apache.hadoop.io.DoubleWritable;
5  import org.apache.hadoop.io.Text;
6  import org.apache.hadoop.mapreduce.Job;
7  import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
8  import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
9
10 import org.apache.hadoop.conf.Configured;
11 import org.apache.hadoop.conf.Configuration;
12 import org.apache.hadoop.util.Tool;
13 import org.apache.hadoop.util.ToolRunner;
```

Implementing the *Tool* Interface and Extending the *Configured* Class.

```
public class AvgWordLength {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.out
                .printf("Usage: AvgWordLength <input dir> <output dir>\n");
            System.exit(-1);
        }
```

```
public class AvgWordLength extends Configured implements Tool {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.out
                .printf("Usage: AvgWordLength <input dir> <output dir>\n");
            System.exit(-1);
        }
```

Implementing the *Tool* Interface and Extending the *Configured* Class.

- main() calls *ToolRunner.run*.
- Change the original main() into something else.

```
public class AvgWordLength extends Configured implements Tool {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.out
                .printf("Usage: AvgWordLength <input dir> <output dir>\n");
            System.exit(-1);
        }
```

```
public class AvgWordLength extends Configured implements Tool {

    public static void main(String[] args) throws Exception {
        int exitCode = ToolRunner.run(new Configuration(), new AvgWordLength(), args);
        System.exit(exitCode);
    }

    public int run(String[] args) throws Exception {
```

Implementing the *Tool* Interface and Extending the *Configured* Class.

- main() calls *ToolRunner.run*.
- Change the original main() into something else.

```
14        if (args.length != 2) {
15            System.out
16                .printf("Usage: AvgWordLength <input dir> <output dir>\n");
17            System.exit(-1);
18        }
19
20        //create new job object and identify the jar which contains mapper and reducer
21        Job job = new Job();
```

```
if (args.length != 2) {
    System.out.printf("Usage: %s [generic options] <input dir> <output dir>\n",getClass().getSimpleName());
    return -1;
}

//create new job object and identify the jar which contains mapper and reducer
Job job = new Job(getConf());
```

Configuration conf = new Configuration();
Job job = Job.getInstance(conf, “Average Word Length”);

Replace Job job = new Job(getConf()); in Hadoop 2.x or above.

```java
package solution;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class ToolRunnerWC extends Configured implements Tool {

    public static void main(String[] args) throws Exception{
        int exitCode = ToolRunner.run(new Configuration(), new ToolRunnerWC(), args);
        System.exit(exitCode);
    }

    public int run(String[] args) throws Exception {
      if (args.length != 2) {
        System.out.printf("Usage: %s [generic options] <input dir> <output dir>\n",getClass().getSimpleName());
        return -1;
      }

    // Create new job object and indentify the jar which links to the mapper and the reducer
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Tool Runner Word Count");
    job.setJarByClass(ToolRunnerWC.class);
    job.setJobName("Word Count");

    // Input/Output file paths
    FileInputFormat.setInputPaths(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    // Mapper and Reducer classes
    job.setMapperClass(WordMapper.class);
    job.setReducerClass(SumReducer.class);

    System.out.print("Programmer: Pei-Wei Tsai\nStudent ID: 92144979\n");

    // Set output key/value classes
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    // Start the MapReduce job and wait for it to complete. Success returns 0.
    boolean success = job.waitForCompletion(true);
    return success ? 0 : 1;
  }
}
```

IMPLEMENTING THE *TOOL* INTERFACE AND EXTENDING THE *CONFIGURED* CLASS.

The completed driver code.

ToolRunner Command Line Options

- Command line options are commonly used to specify Hadoop properties using the –D flag
  - Overrides any default or site properties in the configuration.
  - But will **not** override those set in the driver code.

- Command examples:
  - hadoop jar *yourjar.jar YourDriver* **–D mapred.reduce.tasks=10** *inputdir outputdir*
  - hadoop jar *yourjar.jar YourDriver* **–conf** ***[XML configuration file]*** *inputdir outputdir*

SETTING UP AND TEARING DOWN MAPPERS AND REDUCERS

Why Do We Need a Tearing Down Flow?

- It is very common that you want to execute some codes before the map or reduce method is called for the first time.

- The *setup* method run before the map or reduce method is called for the first time.

```
public void setup(Context context)
```

Why Do We Need a Tearing Down Flow?

- Similarly, you can also do something after the mapper or reducer is executed.

- The *cleanup* method is called after the mapper or reducer terminates.

```
pulbic void cleanup(Context context) throws IOException, InterruptedException
```

Passing Parameters

For the Driver Class

```
public class MyDriverClass {
    public int main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        conf.setInt ("paramname", value);
        Job job = new Job(conf);
        ...
        boolean success = job.waitForCompletion(true);
        return success ? 0 : 1;
    }
}
```

For the Mapper Class

```
public class MyMapper extends Mapper {
    public void setup(Context context) {
        Configuration conf = context.getConfiguration();
        int myParam = conf.getInt("paramname", 0);
        ...
    }

    public void map...
}
```

DECREASING THE AMOUNT OF INTERMEDIATE DATA WITH COMBINERS

The Combiner

In general, Mapper program produces:

- Large amounts of intermediate data.
  - The data must be passed to the reducers.
    ➔ Large network traffic.

Solution:

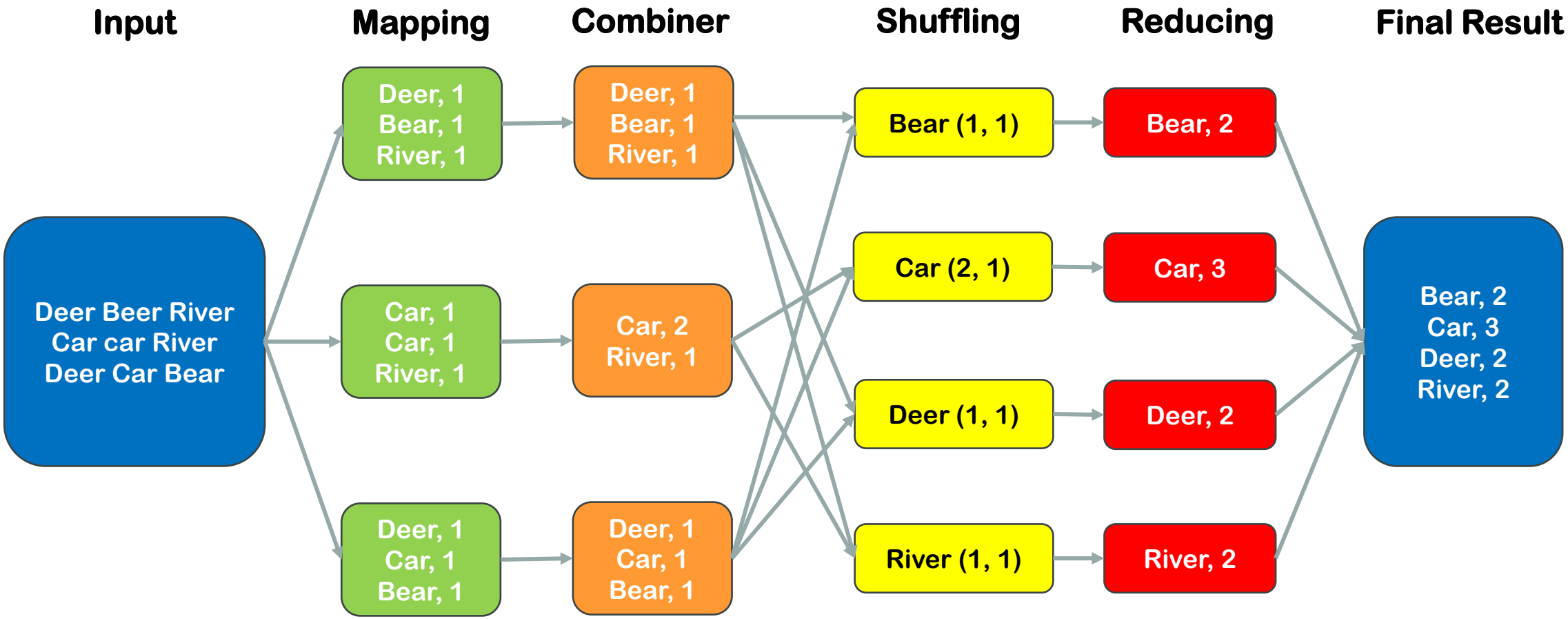
- Specify a *Combiner*.
  - A mini-reducer.
  - Runs locally on a single Mapper's output.
  - Output from the Combiner is sent to the Reducers.

The Combiner

Combiner and Reducer codes are often identical

- Technically, this is possible if the operation performed is commutative and associative.
- Input and output data types for the Combiner/Reducer **must be** identical.

Combiner – Local Reduce



Writing a Combiner

- The combiner uses the same signature as the Reducer.
  - Input: <key, value> pairs.
  - Output: 0 or multiple <key, value> pairs.
  - It is called as a function in the class.

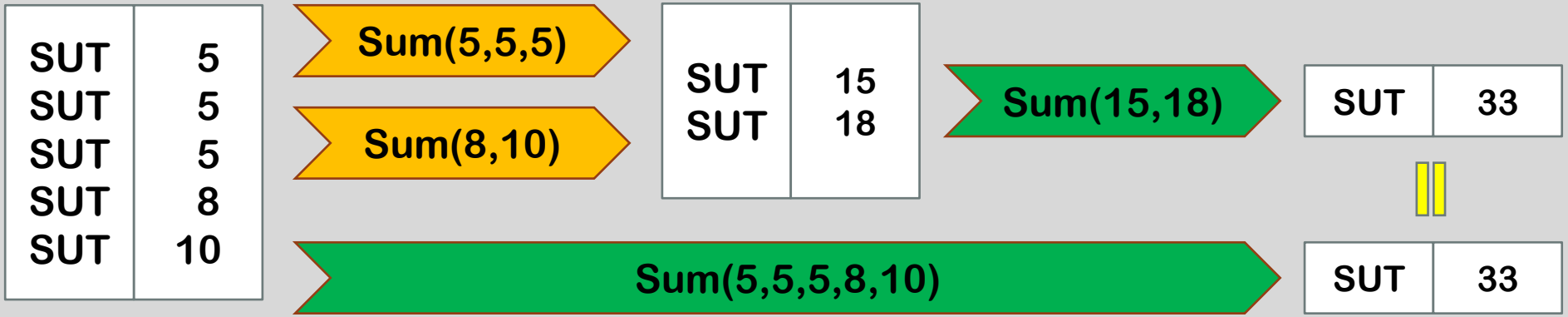
reduce(inter_key, [v1, v2,..]) → (result_key, result_value)

Question

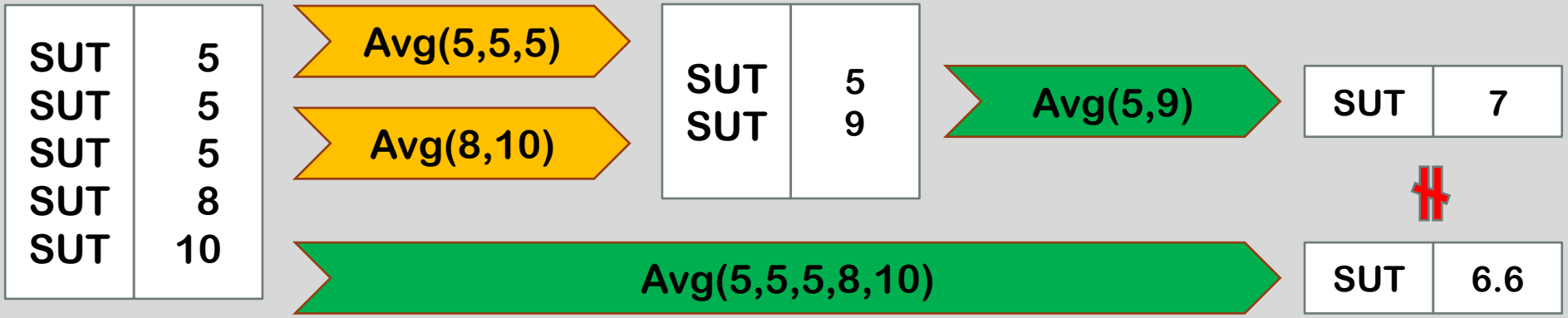
- Can the Reducer be used as the Combiner?
- **Only in some cases**, the Reducer can be used as the Combiner.

Combiners and Reducers

- If the operation is associative and commutative. (Ex: SumReducer)



- If the operation is not associative and commutative. (Ex: AverageReducer)



Specifying a Combiner

```
job.setMapperClass(YourMapper.class);
job.setReducerClass(YourReducer.class);
job.setCombinerClass(YourCombiner.class);
```

The command on the left helps you to specify the Combiner class to be used in your MapReduce code in the **driver.**

The input/output data type for the Combiner and the Reducer for a job must be identical.

The Combiner may run **once, or more than once**, on the output from any given Mapper.

Accessing HDFS Programmatically

Moving from command-line shell to programmatically accessing HDFS.

- Useful if your code needs to read/write "side data" in addition to the standard MarReduce inputs and outputs.
- For programs outside of Hadoop to read the results of MapReduce jobs.

Beware: HDFS is NOT a general-purpose filesystem.

- Files cannot be modified once they have been written.

Hadoop provides the FileSystem abstract base class

- Provides an API to generic file system.
  - Could be HDFS, your local file system, other file systems such as Amazon S3.

FileSystem API

- In order to use the FileSystem API, retrieve an instance of it.

```
Configuration conf = new Configuration();
FileSystem fs = FileSystem.get(conf);
```

- The *conf* object has read in the Hadoop configuration files, and therefore knows the address of the NameNode.

- A file in HDFS is represented by a *Path* object.

```
Path p = new Path("/path_to_your_file");
```

FileSystem API: Directory Listing

- Get a directory listing.

```
Path p = new Path("/path_to_your_file");

Configuration conf = new Configuration();
FileSystem fs = FileSystem.get(conf);
FileStatus[] fileStats = fs.listStatus(p);

for (int i = 0; i < fileStats.length; i++) {
    Path f = fileStats[i].getPath();

    ...
}
```

FileSystem API: Writing Data

- Write data to a file.

```
Configuration conf = new Configuration();
FileSystem fs = FileSystem.get(conf);

Path p = new Path("/path_to_your_file");

FSDataOutputStream out = fs.create(p, false);

// write some raw bytes
out.write(getBytes());

// write an int
out.writeInt(getInt());

...

out.close();
```

USING THE DISTRIBUTED CACHE

The Distributed Cache

A common requirement is for a Mapper or Reducer to need access to some “side data” such as look up tables, dictionaries, and standard configuration values.

Distributed Cache provides an API to push data to all slave nodes.

- Transfer happens behind the scenes before any task is executed.
- Data is only transferred once to each node.
- Distributed Cache is **read-only**.
- Files in the Distributed Cache are automatically deleted from slave nodes when the job is finished.

Without a cache, the values can still be read directly from HDFS in the *setup* method.

Using the Distributed Cache – Method I

1. **Place the files into HDFS**
2. **Configure the Distributed Cache in your driver code.**
   - **.jar files added with *addFileToClassPath* will be added to your Mapper or Reducer's classpath.**
   - **Files added with *addCacheArchive* will automatically be dearchived/decompressed.**

```
Configuration conf = new Configuration();
DistributedCache.addCacheFile(new URI("/myapp/lookup.dat"),conf);
DistributedCache.addFileToClassPath(new Path("/myapp/mylib.jar"),conf);
DistributedCache.addCacheArchive(new URI("/myapp/map.zip"),conf);
DistributedCache.addCacheArchive(new URI("/myapp/mytar.tar"),conf);
DistributedCache.addCacheArchive(new URI("/myapp/mytgz.tgz"),conf);
DistributedCache.addCacheArchive(new URI("/myapp/mytargz.tar.gz"),conf);
```

Using the Distributed Cache – Method II

- Using ToolRunner and add files to the Distributed Cache directly from the command line when you run the job.
  - Note: You don't need to copy the files to HDFS first if you chose to use Method II.
- Use the –files option to add files

```
hadoop jar myjar.jar MyDriver -files file1, file2, file3, ...
```

- The –archives flag adds archived files, and automatically unarchives them on the destination machines.
- The –libjars flag adds jar files to the classpath.

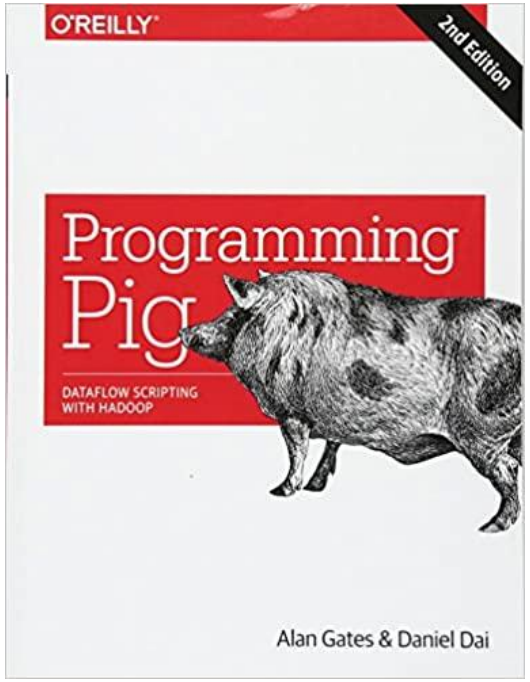
Accessing Files in the Distributed Cache

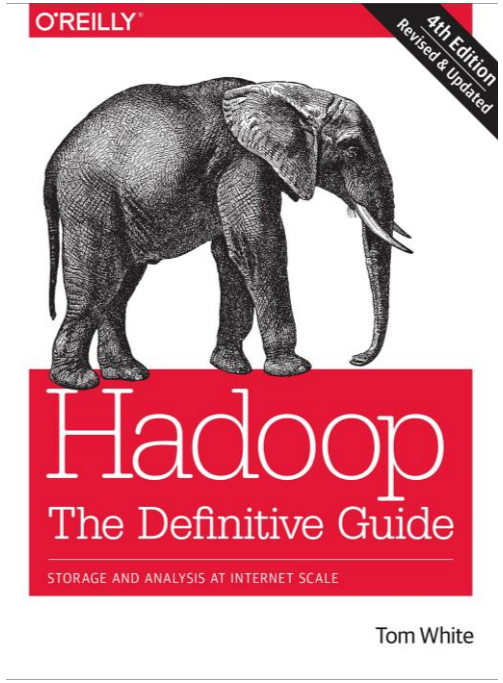
- Files added to the Distributed Cache are made available in your task's local working directory.
- Access them from your Mapper or Reducer the way you would read any ordinary local file.

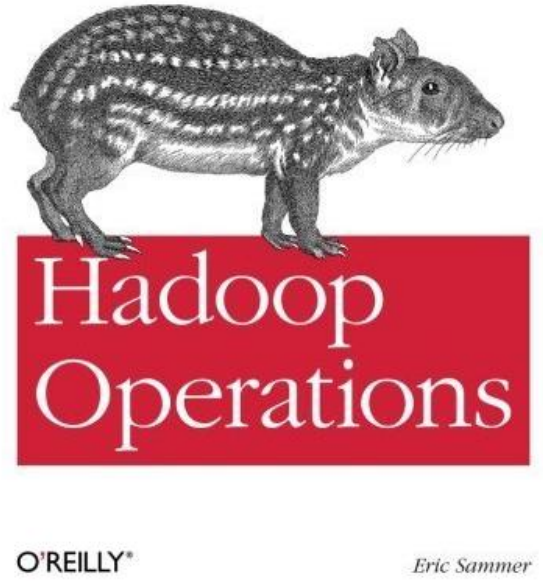
```
File f = new File("file_name");
```

Reusable Classes for the New API

- *Org.apache.Hadoop.mapreduce.lib.*/** packages contain a library of Mappers, reducers, and Partitioners supporting the new API.
- Refer to the Javadoc for classes available in your version of Cloudera Documentation Hadoop (CDH).
  - Available classes vary greatly from version to version.
- Example classes
  - *InverseMapper* – Swaps keys and values.
  - *RegexMapper* – Extracts text based on a regular expression.
  - *IntSumReducer*, *LongSumReducer* – Add up all values for a key.
  - *TotalOrderPartitioner* – Reads a previously-created partition file and partitions based on the data from that file. → Sample the data first to create the partition file. Allows you to partition your data into *n* partitions without hard-coding the partitioning information.







Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
  - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed.., Sebastopol, California: O'Reilly, 2015.
  - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
  - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017

Unit Summary

- MapReduce Code Explanation.